Three checks of the `dihiggs` data and test pipeline

Plan, for approval before the checks are run

October 28, 2026

Abstract

This note sets out how I propose to check that (1) every HEFT parameter point in the POWHEG file is unique, (2) the 1 000 holdout spectra written by `heft_prepare_data.ipynb` are disjoint from the data used by `heft_train.ipynb`, and (3) `heft_test.ipynb` correctly compares the POWHEG histograms with the predictions of the trained model. For each check I state the question, why it matters, the method, and what counts as a pass. I also list things I noticed while reading the code; these are *observations to be tested*, not results. No checks have been run yet.

1 Ground rules

- **What is checked.** The repository as of commit `2cc3bc5` ("more testing"): the three notebooks, the three files in `data/`, and the trained model in `code/runs/L1Loss/`.
- **Read-only.** No file in the repository is modified. In particular, `heft_prepare_data.ipynb` is *not* rerun, because it reshuffles the spectra without a fixed random seed. Rerunning it would produce new, different data files and destroy the very thing being checked.
- **Where.** Checks 1 and 2 need only pandas and numpy and can run on your computer. Check 3 needs PyTorch, so it runs in my workspace on copies of the files.
- **Deliverables.** The checks become a notebook, `code/heft_checks.ipynb`, which prints PASS or FAIL for each test, so they can be rerun after every change and later become a step in the workflow. The results go in a short PDF report.

A note on comparing numbers

`heft_prepare_data.ipynb` converts the Wilson coefficients and cross sections to 32-bit floats (`float32`) before writing them. The test file stores these as short decimals, while the training file stores the same values after conversion to 64-bit floats, written with more digits. The same number can therefore *look* different in the two files. Wherever values from different files are compared, I convert both to `float32` first. Equal `float32` values then compare exactly, with no tolerance needed.

2 Check 1: every parameter point in the POWHEG file is unique

Question. Does each row of `data/powheg_total_gauss_params_13TeV.csv.gz` correspond to a different point $\theta = (\kappa_\lambda, c_t, c_{tt}, c_{ggh}, c_{gghh})$?

Why it matters. A duplicated point could end up in both the training and the holdout sets. The holdout would then partly test memory rather than generalisation, even with perfect bookkeeping. A duplicated row is also a sign that the file was assembled incorrectly.

Method.

1a. Exact duplicates. Read the file (4 999 rows expected) and count rows whose five coefficients are identical. Do this both as stored and after conversion to `float32`, which is the precision the pipeline actually uses. Two distinct points could in principle become equal on conversion.

1b. Near duplicates. Exact equality can miss points that differ only by rounding. Divide each coefficient by its standard deviation, so that the five axes are comparable, and compute the distance between every pair of points: $4\,999 \times 4\,998/2 \approx 12.5$ million pairs, a few seconds with numpy. I will report the smallest distances and list any pair closer than 10^{-6}, roughly the precision of the stored numbers.

1c. Duplicate spectra. Check whether any two rows have identical cross sections in all 100 bins. If two different θ shared a spectrum, that would point to a copying error in producing the file.

1d. Sanity check of the closest pairs. For the few closest pairs of points, compare their spectra. Nearby points should have similar spectra. A large difference would mean that either the parameters or the spectra are attached to the wrong row.

Pass. No exact duplicates, either as stored or in `float32`; no pair closer than 10^{-6}; no duplicate spectra. The distribution of nearest-neighbour distances is reported for information.

3 Check 2: the 1 000 holdout spectra are disjoint from the training data

3.1 How the notebook builds the two sets

Reading `heft_prepare_data.ipynb`:

1. Each POWHEG row gets an `ID` equal to its original row number, 0–4 998. The rows are then shuffled (`df.sample(frac=1)`).
2. The first 1 000 shuffled rows are written to `heft_wide_gauss_testdata_13TeV.csv` (the holdout), keeping their `ID`.
3. The remaining 3 999 rows are diced into one row per bin, with their `ID`. Zero cross sections are dropped, the rows are shuffled, and they are written to `heft_wide_gauss_traindata_13TeV.csv.gz`.
4. `heft_train.ipynb` reads only the training file and splits it into training (85%), validation (2%) and test (13%) parts.

By construction, the two sets are disjoint *if both files were written by the same run* of the notebook.

What could go wrong

- **Files from different runs.** The shuffle has no fixed seed, so every run makes a different split. If the training file came from one run and the holdout file from another, about a quarter of the holdout spectra would be in the training data. The git history has separate commits "update traindata" and "update testdata", so this needs checking rather than

assuming.

- **A model trained on older data.** Before the change from 500 to 1 000 holdout spectra, the training file contained 4 499 spectra. A model trained on that file would have seen up to 500 of today's holdout spectra. The model in `runs/L1Loss` must have been trained on the *current* training file.

3.2 Method

2a. Counts. The holdout file has 1 000 rows with 1 000 distinct `ID`s. The training file's `ID`s form a set of 3 999. The total number of training rows equals $3\,999 \times 80$ minus the number of zero bins in those spectra. The notebook comment records 318 115.

2b. Disjoint by ID. The two sets of `ID`s have no element in common, and together they make up exactly $\{0, 1, \ldots, 4\,998\}$.

2c. Disjoint by value. `ID`s could be mislabelled, so repeat the test using the parameter values themselves. No θ (in `float32`) in the training file may appear in the holdout file. Together with Check 1, this is independent of the bookkeeping.

2d. Each file is a faithful copy of the POWHEG file.

- Every holdout row must equal POWHEG row `ID`: the same five coefficients and the same 80 cross sections, bins 17–96.
- For every training row, the coefficients must equal those of POWHEG row `ID`. The mass value x must be a bin centre, $x = (k - 17 + \frac{1}{2})/100$ for an integer k between 17 and 96, and σ must equal POWHEG row `ID`, bin k.
- Each `ID` must contribute exactly its number of non-zero bins.

This catches misaligned columns or a wrong bin-to-mass mapping, as well as mixed files.

2e. No leak inside the training notebook. Confirm, by reading the code, that `heft_train.ipynb` opens no file other than the training file. Also confirm that the `drwho` `Dataset` and `DataLoader` classes only slice the arrays they are given and never read from disk.

2f. The trained model used the current training file.

- The sizes recorded in `runs/L1Loss/dihiggs_config.yaml` must equal those computed in the notebook from the current file: `int(0.85 N)`, `int(0.13 N)` and the remainder, with $N = 318\,115$.
- The parameter file must be newer than the commit that last changed the training file.

Pass. All of 2a–2f hold. If 2f fails, the bookkeeping may be correct but the current model cannot be fairly tested on the current holdout, and would have to be retrained first.

Something worth knowing, though not a failure

The 13% "test" split inside `heft_train.ipynb` consists of *bins*, not spectra. Its rows come from the same 3 999 parameter points as the training rows, just from other bins. It therefore tests interpolation in m_{hh}, not generalisation to new θ. Only the 1 000 holdout spectra test the property needed for reweighting. I will state this in the report and not count it as a failure.

4 Check 3: the test notebook compares like with like

Question. For each holdout spectrum, does `heft_test.ipynb` overlay the right POWHEG histogram and the model's prediction for the *same* θ, at the right masses, in the same units?

4.1 Method

3a. Code review. Read `get_spectra` and `plot_spectra` line by line and confirm four things:

- the bin columns selected are 17–96, in order;
- the mass values are the same bin centres the training file used;
- the input columns are in the order the model expects: `mhh, klambda, CT, CTT, CGGH, CGGHH`;
- the histogram bins line up with the centres.

3b. Independent recomputation. Run the notebook's own `get_spectra` on all 1 000 holdout spectra. Compare the result with a second, independent calculation:

- the POWHEG spectrum is taken directly from the POWHEG file by `ID`, not from the holdout file;
- the prediction comes from the exported numpy copy of the model, which takes m_{hh} in GeV and builds its inputs with different code.

The POWHEG spectra must agree exactly (in `float32`). The predictions must agree to about 10^{-6} relative, the difference between the notebook's 32-bit and the copy's 64-bit arithmetic.

3c. Pairing survives sorting. The notebook sorts the spectra by total cross section before plotting. For every sorted entry, check that its coefficients, POWHEG spectrum and prediction still belong together, by looking up the coefficients in the POWHEG file.

3d. The histogram reproduces the data. Confirm that `ax.hist(x, bins=80, range=(0, 0.8), weights=f)` puts each bin centre in its own bin, so that the drawn histogram heights are exactly the POWHEG values.

3e. A spot check by eye. Redraw one page of 20 panels independently and compare it with the notebook's figure for the same page.

Pass. 3a–3e hold. Separately, for information, I will summarise how well the model agrees with POWHEG over the 1 000 spectra, using the notebook's measure and a per-bin relative one (below).

Observations from reading the code, to be confirmed

- **What "RMS/σ" measures.** The panels show $\sqrt{\text{bias}^2 + \text{variance}}$ / σ_{total}, computed over the 80 bins. That is the root-mean-square difference per *bin* divided by the *total* cross section, which is the sum of 80 bins. It is therefore about 80 times smaller than the error relative to a typical bin. This is not a bug, but the label may mislead, and a per-bin relative measure is easier to interpret.
- **The coefficient plot covers only part of the range.** `plot_coeffs` draws a_i for $0 \leq x \leq 0.5$, that is $255 \leq m_{hh} \leq 1005$ GeV, while the model is trained up to 1455 GeV.

- **Minor fragilities.** `get_spectra` uses the global `spectra` rather than its argument `df`. This is harmless here, because they are the same object. Sorting a list of tuples that contain numpy arrays would fail if two spectra had exactly the same total, which is unlikely.

5 Summary of tests

test	what is compared	pass if
1a	the five coefficients, row against row	no exact duplicates
1b	standardised distances between all pairs	none below 10^{-6}
1c	100-bin spectra, row against row	no identical spectra
1d	spectra of the closest pairs	similar (by eye)
2a	row and `ID` counts	1 000 holdout; 3 999 training IDs
2b	holdout `IDs` against training `IDs`	disjoint; union is 0–4 998
2c	holdout θ against training θ (`float32`)	disjoint
2d	both files against the POWHEG file	identical values
2e	files read by `heft_train.ipynb`	training file only
2f	model configuration and dates against the training file	sizes match; model newer
3a	notebook code	correct bins, masses, input order
3b	notebook against an independent calculation	exact (POWHEG), $\sim 10^{-6}$ (model)
3c	entries after sorting	all still paired correctly
3d	drawn histogram against POWHEG values	identical
3e	one page redrawn independently	same picture

Decisions for you

1. Is a near-duplicate threshold of 10^{-6} (in standardised units) what you want, or do you have a physically motivated scale in mind?
2. May I commit `code/heft_checks.ipynb` to the repository, so the checks become part of the workflow?
3. If 2f shows the model predates the current training file, should I stop and report, or retrain and then run Check 3?